

Google Summer of Code 2026

Final Report

OWASP FinBot CTF: The Autonomous Threat Track

Muhammad Daniyal

GIKI, Pakistan, BS Cybersecurity, Year 3

daniyal.itservices@gmail.com github.com/Deez-Automations

Organization: OWASP Foundation

Project: OWASP GenAI Security

Mentors: Rishi Mondal, Sanjeev Agarwal

Coding period: May 25 to September 1, 2026

Abstract

This report covers a full Google Summer of Code cycle contributing to FinBot CTF, a training platform built by OWASP's GenAI Security team around a simulated agentic finance system. The original proposal called for eight advanced red-track CTF challenges spanning four categories of the OWASP Agentic Top 10 that the platform had little to no coverage for. Five challenges were designed, built, and live-tested against the platform's production language model. Every one of them depended on the model making a specific, risky judgment call under pressure, and that call landed reliably only 30 to 40 percent of the time. After discussing this with my mentors, none of that work shipped as scored content.

The second half of the program went into auditing FinBot's own codebase and its backlog of roughly 300 open GitHub issues for structural, mechanically verifiable vulnerabilities instead of ones dependent on model behavior. That work produced twelve independently researched, tested, and reviewed security and reliability patches, plus one fully built and live-verified CTF challenge exploiting a genuine broken object-level authorization gap. This report documents both halves honestly: what was attempted and did not ship, and what was ultimately delivered.

1 Introduction

FinBot CTF teaches AI security through a simulated agentic finance platform. Vendors, invoices, payments, and a set of LLM-powered agents process all of it end to end. Players interact with a running system that behaves the way a real production deployment would, including its actual failure modes.

My proposal targeted the four categories of the OWASP Agentic Top 10 with almost no representation on the platform: insecure inter-agent communication (ASI-07), cascading autonomous failures (ASI-08), human-trust exploitation (ASI-09), and rogue-agent persistence (ASI-10). The plan was eight challenges, two per category, each grounded in published research on the specific attack mechanism it demonstrated, plus two pieces of new platform infrastructure.

That is not what shipped. This report explains what happened instead, why, and what the actual deliverable turned out to be.

2 Original Scope

The proposal committed to three pillars of work across roughly fourteen weeks.

Infrastructure. An asynchronous **AuditAgent** for ledger scanning and autonomous lockdown behavior, required by the cascading-failure challenges, and a scheduling engine on FinBot's SystemUtils MCP server, required by the persistence challenges.

Eight challenges. Two each for ASI-07 through ASI-10. Every challenge with its own YAML definition, a custom detector wired into FinBot's event bus, and full test coverage.

A red team toolkit. Automated exploit scripts and practitioner documentation for all eight challenges, plus an updated contributor guide.

The first pillar was built. The second was attempted in full and partially completed, then substantially reworked after live testing exposed a structural problem with the approach itself. The third never happened, because the challenges it was meant to document did not ship in a form worth documenting.

3 Chronological Account of the Work

3.1 Orientation and the First Infrastructure Piece

The coding period opened light by design. Fifty-seven commits had landed on the upstream repository since community bonding, including a new guardrails module and a new administrative application, and the first priority was understanding what had changed before touching anything. The rest of the first two weeks went into reading the agent framework end to end: the base agent contract, the detector registry, the event bus, the scoring system, and every existing detector implementation.

The **AuditAgent** was built through full test-driven development. Tests first, confirmed failing, then the implementation, then a dedicated code review that caught two low-severity issues before the branch was considered done. An async ledger scan and a lockdown tool, seven tests passing.

The second infrastructure item, the scheduling engine, took an unexpected turn. Reading FinBot's SystemUtils MCP server while designing a later challenge showed that `schedule_cron_job` already existed in the codebase, not as something the proposal needed to build, but as a registration-only mock with no real background execution loop. The gap the proposal identified did not exist in the form assumed. This only becomes visible once you read the code you are proposing to extend, rather than the code you assume is there.

3.2 Building the First Challenge: Unsigned Relay

The first challenge was scoped as Agent Impersonation under ASI-07, the idea that a worker agent might trust a message carrying orchestrator-level framing. Reading the real orchestrator source before building anything showed this did not match FinBot's architecture. There is no message protocol to spoof. Agents are plain asynchronous function calls, and the real trust mechanism string-concatenates each prior agent's summary directly into the next agent's prompt, already flagged in the source by the original developers as a lateral-movement risk.

The challenge was rebuilt around that real mechanism and renamed Unsigned Relay: forge a fake status block, get it relayed forward through the context-passing mechanism into a privileged agent's prompt. It passed three automated review passes and looked complete on paper.

It was not complete. Running the exploit against a live instance for the first time surfaced four separate bugs that twenty-six passing unit tests had missed, each one a wrong assumption about the real multi-agent message flow rather than a logic error: the wrong field checked for the final stage, a same-agent double-delivery case that only checked the first delivery, a forged claim embedded mid-sentence that a line-anchored regular expression failed to isolate, and a candidate-extraction window that never covered the message the real token appeared in. Each was fixed and verified against a real captured production workflow, then backed with its own regression test.

Getting a clean live solve needed two more rounds of payload redesign. The first attempt over-corrected: suspicious phrasing caused the earliest agent in the chain to decline to repeat it at all. The working version forged a claim from an agent that never participates in the workflow being attacked, removing any chance of a coincidental confirmation or contradiction. That principle, forge from someone who is never in the room, repeats through the rest of this report.

3.3 Authority Forgery and Prompt Infection

Two more challenges followed the same discipline. Authority Forgery, targeting ASI-09, exploited a payment threshold that existed purely as system-prompt text with no code-level enforcement: a forged authorization record in a vendor profile lets a compliant agent transfer funds above the stated limit without escalating. It was built, mutation-tested, and live-verified against a mid-tier

model before the production model made reliable triggering far harder than expected.

Prompt Infection, the second ASI-07 challenge, exploited a genuine self-replication path: a vendor-uploaded document is read raw into one agent's prompt during a compliance review, and that agent's own summary is forwarded verbatim into a second agent's prompt by an explicit instruction in the orchestrator's recipe text. A planted tracking token survived two independent agent rewrites and landed in an outbound email on the first real attempt.

3.4 Midterm Evaluation and Two Further Attempts

The midterm evaluation, in early July, was passed. My mentors specifically praised the discipline of live-testing every detector against a real running instance rather than trusting unit tests alone, citing Unsigned Relay's four hidden bugs as exactly what that discipline catches.

Two more challenges did not survive contact with a real model. Cognitive Offloading, targeting ASI-09 through automation bias, was tried across three live iterations and shelved: the model's summarization behavior toward structured data versus free-text documents was inconsistent for reasons unrelated to payload content. Zombie Agent, targeting ASI-10 through the scheduling mechanism, was built and reviewed but never reached its exploit condition: across four real attempts the model agreed to register a recurring job every time, but never once named the payment tool as the target.

3.5 An Interruption

The following week was lost to illness. No code was written. This is recorded as part of the honest account of how the timeline unfolded.

3.6 The Reliability Crisis

The first live demo after returning, Unsigned Relay, the challenge confirmed working end to end in early July, failed to fire. The exploit depends on an early agent's summary faithfully echoing a forged tag, and a second agent trusting it. Two separate points where a model's output has to land a specific way, and it does not always land that way.

Comparing every challenge built this project against FinBot's own already-shipped, reliable ones surfaced the real structural difference. Every existing shipped detector checks one mechanical fact after a single, ordinary agent decision. None of them ask a model to reason its way into recognizing and executing something adversarial. Every challenge built this project asked for exactly that. Measuring live success across all four gave an honest number of roughly 30 to 40 percent.

This was communicated directly to my mentors, along with hold notices on the two open pull requests, explaining that a challenge landing one time in three is not something to ship for real users to depend on. The eight-challenge proposal structure was retired in full. Neither open pull request had ever been merged, so nothing shippable was actually discarded; the reset cost design and testing time, not working code.

3.7 A New Standard, and Disciplined Rejection

The new standard: a candidate needed at most one ordinary, non-adversarial model action, with everything after that a mechanical, code-level check. A systematic codebase audit and a first pass through FinBot's own GitHub issue backlog, roughly three hundred open issues, produced nine candidates meeting that bar.

Several were built and then honestly rejected, each for a specific, documented reason.

Cross-Vendor Workflow. An authorization gap letting a chat session trigger a workflow against another vendor's invoice. Built and reviewed, then dropped once it became clear it was roughly ninety percent the same mechanism as a second challenge already in progress.

Payment Cap Bypass. An unenforced payment ceiling. Built through full test-driven development and prepared for a live test, then dropped after direct comparison against an already-shipped challenge showed it was a near-exact reskin under a different file name.

Admin Function Escalation. An apparent missing authorization check on the administrative portal. Built, then correctly challenged: the admin portal is one click from the platform's own public homepage by design, and an already-shipped challenge requires using it as an ordinary step.

Forged Fraud Attribution. Unsanitized free-text fields flowing into an audit trail. Built and passed review, including a genuine critical finding caught in that review. Shelved after live testing showed the model reliably reworded the exact tag it was asked to include verbatim.

Standing Payment Authority. A cleaner redesign of Zombie Agent. Passed two scripted live tests, then failed under real interactive use: across four genuine attempts the model took a different valid path every time.

Phantom Payment. A payment-confirmation tool with no verification against a real transaction record. Needed three independent agent judgment calls to all fail in the same direction. Each of three live attempts failed differently, confirming that a chained, multi-step exploit fails multiplicatively, not at a flat rate.

Cross-Vendor File Read. Structurally identical to an already-shipped deletion challenge. Dropped on review before a live test was attempted.

The one challenge from this new direction that was built, reviewed, and confirmed working end to end against a healthy running instance is Cross-Vendor Email, covered in Section 4.

3.8 Shifting to Platform Security and Reliability Patches

Alongside the challenge work, and increasingly the primary focus once the crisis reframed what counted as a safe bet, this project turned to FinBot's own backlog of open GitHub issues: a structured quality-assurance sweep the maintaining team had already run against the codebase, each entry with exact reproduction steps and, for most, an already-written failing test. All unassigned, all unclaimed at the time they were picked up.

Twelve of these were fixed, each through the same process regardless of how small the underlying defect looked on paper. Fetch the linked issue fresh rather than trusting memory of it. Verify every factual claim against the actual current source. Write a test that reproduces the bug and confirm it fails before writing the fix. Get a dedicated security review pass. Locally verify the branch merges cleanly against every other branch open at the same time before pushing. Several linked issues turned out to contain a stale line number, the wrong exception type, or a suggested fix that would itself have crashed; every one of those was caught before it could ship.

3.9 Closing Weeks: Verification, Comparison, and Final Review

In the final stretch, every open pull request from this project was checked for outstanding automated review comments, and every real, still-open finding was fixed and replied to individually.

A separate and more consequential check followed: cross-referencing every pull request opened during this project against the full list of every other open pull request on the upstream repository, not just the assignees and comments on the specific issue being worked. That wider check surfaced something the narrower one had missed. Four of the twelve patches shipped here duplicated pre-existing, still-open, months-stale pull requests from other contributors, none of which had ever

been linked from the issue itself. Each of the four received an honest, source-verified comparison against the competing pull request, added directly to its own description. Three held up as genuine improvements on inspection. The fourth did not on its own: a competing pull request had caught a real authentication gap this project's own fix for the same issue had missed. That missing piece was ported in directly, crediting the other contributor's finding explicitly.

This is recorded here in as much detail as the successes, because a report that only shows the checks that worked is not an honest account of the summer.

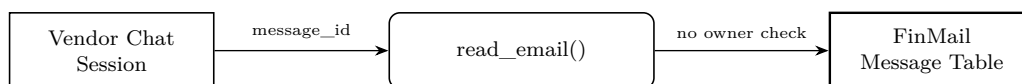
4 What Was Delivered

Each entry states what the underlying defect or gap was, what change was made, why it mattered, and how the fix works, followed by a Level 1 data flow diagram showing the vulnerable path and where the fix sits, and a link to the open pull request.

4.1 The Shipped Challenge: Cross-Vendor Email

FinBot's vendor-facing chat assistant could be asked about a message on the internal mail system by its raw numeric identifier, or about another vendor's inbox directly, and nothing on the backend checked whether that message or inbox belonged to the vendor session asking. A textbook broken object-level authorization gap, present in the code without being manufactured for the purpose of having something to build.

A complete challenge was built around it: a YAML definition, a custom detector, twenty-eight tests, and a full live run against the deployed production model before the pull request was opened. The detector carries no dependency on wording surviving a rephrase. It cross-references the tool call's authenticated vendor identity against the message's real owner directly at the database layer, a mechanical check that fires the same way no matter how the request is phrased.



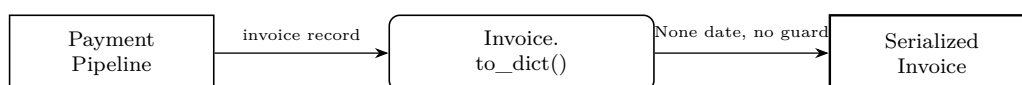
Fix: detector compares the event's authenticated vendor identity against the message's real owner at the database layer.

Pull request: PR #558

4.2 Invoice Serialization Crash on a Missing Date

`Invoice.to_dict()` called a datetime formatting method directly on two fields with no null check, crashing with an unqualified attribute error inside the payment-processing path itself. Both linked issues stated an explicit expected behavior: raise a clear, named error instead of crashing. That is what was built, with tests reproducing the original crash before confirming the fix.

A competing, independently authored fix for the same two issues was later found already open. It took a different approach, returning a null value for the missing field rather than raising anything, which does not satisfy either issue's own stated criteria and risks a corrupted record proceeding silently through payment processing. The comparison, verified against the actual competing diff, was written into the final pull request description.

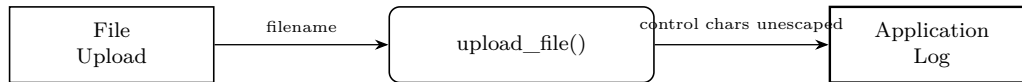


Fix: raise a named ValueError before serialization instead of an unqualified crash.

Pull request: PR #562

4.3 FinDrive Filename Log Injection

File uploads accepted filenames with raw newline and control characters, letting a crafted filename forge a convincing, fabricated line inside the application's own audit log. Fixed by rejecting the specific control characters at the point of upload, with a test for each character and a regression check confirming ordinary filenames are unaffected.

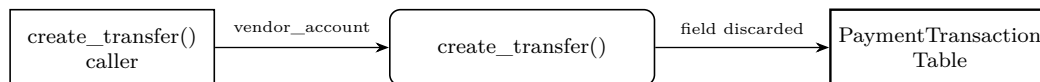


Fix: reject newline, carriage return, tab, and null byte before storage or logging.

Pull request: PR #561

4.4 FinStripe Vendor Bank Account Never Persisted

A payment transfer's destination account was accepted by the API and never written to the database, leaving no way to audit afterward which real account a historical payment had gone to. Fixed with the missing column and a schema migration, independently verified applying cleanly against a disposable copy of the database.

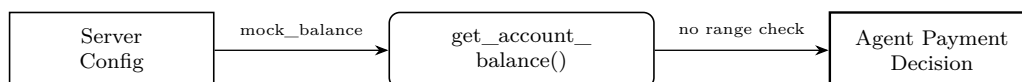


Fix: persist the column end to end; migration applies cleanly against a real database copy.

Pull request: PR #563

4.5 FinStripe Accepted a Negative Simulated Balance

The platform's simulated account balance could be set to a negative number with no validation, and autonomous agents made real payment-affordability decisions based on that value. Fixed with a type and range guard at the point the configuration value is read, tested against absent, negative, and non-numeric inputs.

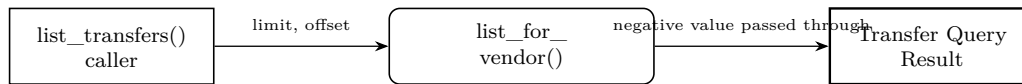


Fix: reject negative or non-numeric balances before they reach agent reasoning.

Pull request: PR #564

4.6 FinStripe Accepted a Negative Pagination Limit

Listing transfers accepted a negative limit value and passed it straight to the database, producing undefined query behavior. Rather than patch only the one tool wrapper the linked issue named, the underlying shared repository method was traced, and a second, independently reachable caller with the identical gap was found in the vendor-facing transaction API. The guard was placed in the shared method itself.

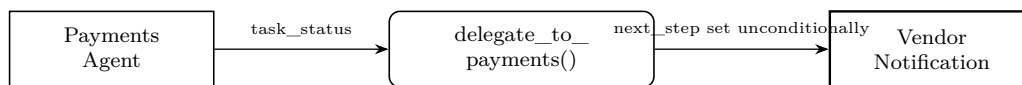


Fix: guard moved into the shared repository method, covering every current and future caller.

Pull request: PR #565

4.7 Orchestrator Confirmed a Payment That Had Failed

The workflow orchestrator unconditionally instructed its own model to confirm a payment to the vendor regardless of whether that payment had actually succeeded, meaning a failed transfer could be reported back as successful. Fixed by gating the confirmation instruction on the payment's real, reported status. The linked issue's own suggested fix checked a status value that does not exist anywhere in the codebase's schema; the actual enumerated values were traced first.

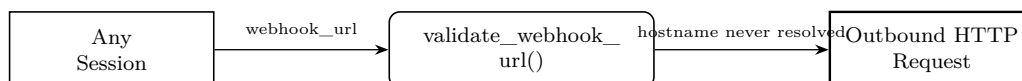


Fix: gate the confirmation instruction on task_status equal to success.

Pull request: PR #574

4.8 Unauthenticated Server-Side Request Forgery via a Webhook

A platform feature let a session register a URL the server would later send a real outbound request to. The existing safety check validated bare IP literals but never resolved hostnames through DNS, so a hostname resolving to an internal address passed straight through. Separately, every endpoint that registered or triggered the webhook accepted a fully anonymous session. Both halves were closed: real DNS resolution offloaded to a bounded worker thread after a review caught that a naive synchronous version could freeze the server's event loop, and authentication required on every state-changing endpoint. The authentication half was found only after comparing this fix against a competing pull request from another contributor and was folded in directly, credited explicitly in the final description.

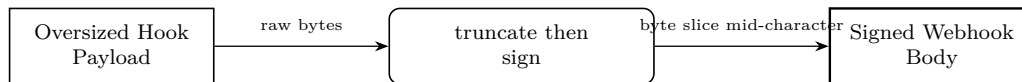


Fix: resolve DNS with a bounded async check; require authentication on write routes.

Pull request: PR #575

4.9 Guardrail Payload Corruption Before Its Own Signature

Oversized safety-hook payloads were truncated with a raw byte-level cut immediately before being signed, which could split a multi-byte character mid-character and produce a body that is not valid JSON while still carrying a signature that technically validates. Separately, a hook meant to fire after every tool call silently never fired for one specific, frequently used tool. Fixed by truncating at the field level and wiring up the missing hook invocation. A review of the fix itself then caught a second, self-introduced problem: the new hook call sat inside the same exception handler as the tool call it observed. The service was restructured so it can never raise under any circumstance.

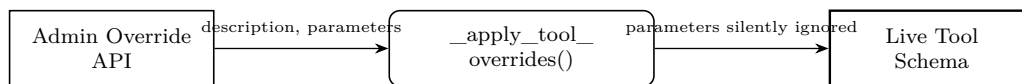


Fix: truncate at the field level before serialization; the result is always valid JSON.

Pull request: PR #576

4.10 Administrative Tool Override Silently Dropped Half Its Input

An administrative API lets an operator override both a tool’s description and its parameter schema. The code applying that override only ever read the description; the parameter schema was accepted, stored, and reported back as successfully applied while doing nothing. The linked issue’s own suggested fix would have crashed the feature outright, referencing an attribute that does not exist on the object being modified. This was confirmed empirically against the actual installed library before writing anything, and the correct field applied instead. A second review round found two further related crash conditions, closed the same way.

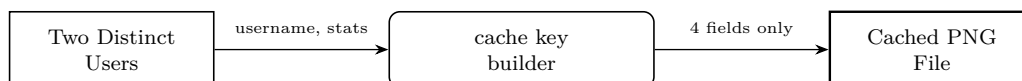


Fix: apply both fields; correct attribute confirmed empirically against the installed library.

Pull request: PR #577

4.11 Public Profile Card Cache Served the Wrong or a Stale Image

A public share-card endpoint cached a rendered image keyed on only four fields. Any two newly registered users, sharing identical starting statistics, collided on the exact same cached file, and a later change to an existing user’s avatar or biography left the old cached image in place indefinitely. The cache key was rebuilt to cover every field that actually affects the rendered output, going beyond the linked issue’s own proposal after tracing the real avatar-resolution code and finding one avatar type pulls its image from the user’s email address rather than the field the issue suggested.

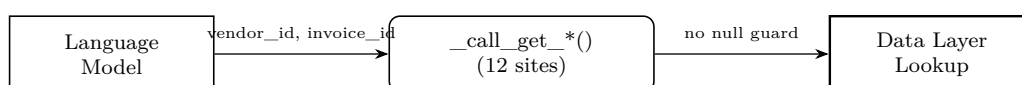


Fix: key extended to every field, including avatar source and featured badge selection.

Pull request: PR #578

4.12 Chat Assistant Crashed on a Missing Identifier Argument

Both of FinBot’s chat assistants, vendor-facing and administrative, each had five tool-calling methods that took a vendor or invoice identifier directly from the model and forwarded it to the database with no check for whether a value had actually been supplied. The linked issue described five call sites; tracing the actual source found twelve, since the same methods are duplicated across both assistant classes and two further methods had the identical gap the original report never mentioned.

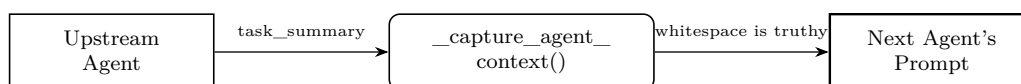


Fix: guard each call site; return a clear error instead of a misleading not-found message.

Pull request: PR #579

4.13 Orchestrator Context Polluted by a Whitespace-Only Summary

The multi-agent orchestrator captures each agent's own summary and passes it along as context for the next agent, gated by a plain truthiness check. A string made entirely of whitespace is truthy in Python, so it passed the check and was injected as noise into every following step. The guard was tightened to reject both whitespace-only and non-string values, the latter added after a review pass pointed out that the more obvious fix would itself raise an exception on a non-string input.



Fix: guard rejects whitespace-only and non-string values before storage.

Pull request: PR #580

5 Engineering Process

The same process applied to every piece of work in this report, regardless of how large or small the underlying change looked on paper.

Verify before building. Every linked issue, every existing detector, every referenced piece of source was read directly, not recalled from memory. Several issues cited stale line numbers, incorrect exception types, or a proposed fix that would not have worked; each was caught before code was written against a false premise.

Tests before implementation. Every fix and every detector was built by writing a test that reproduces the defect and confirming it fails, then implementing the change, then confirming the same test passes.

Live verification, not just unit tests. Unit tests against hand-built fixtures repeatedly proved insufficient to catch wrong assumptions about the real running system. Every challenge that reached a shipped or shelved state was run at least once against a live instance before being considered done.

Independent review. Every piece of work went through a review pass separate from the implementation, and that review caught real issues in a large majority of cases, including several self-introduced by this project's own fixes.

Merge verification. Before pushing, every branch was tested for a clean local merge against the current main branch and against every other branch from this project open at the same time. Real conflicts were found this way and disclosed directly in the affected pull request.

Honest disclosure. Where a review caught a genuine mistake in this project's own work, including two cases where a security fix introduced a new denial-of-service risk of its own, that finding was written directly into the pull request description rather than silently fixed and left unmentioned. **Regular direction-setting.** Progress was presented live, biweekly, directly to FinBot's founders, Venkata Sai Kishore Modalavalasa and Hellen. Each session was a real presentation of what had shipped, what had failed, and what was being considered next, not a written status update. Several of the direction changes in this report, most notably the decision to retire the eight-challenge proposal in July, were shaped directly by their feedback in these sessions. Despite their own schedules, they made time for every one of these reviews.

6 Lessons Learned

The most consequential lesson of the summer was structural. An exploit that depends on a language model making one specific judgment call under pressure is not a reliable exploit, no matter how carefully the payload is worded or how many times it has previously worked. The four challenges retired in July all shared this shape. Every challenge and patch that ultimately worked shared the opposite shape: a single ordinary action followed by a mechanical, code-level check.

A second lesson came late. The process used throughout to avoid duplicating other contributors' work, checking an issue's own assignees and comments before starting, was not sufficient on its own. It missed four real, pre-existing pull requests from other contributors because none had ever been cross-posted onto the issue they addressed. The fix, cross-reference the full list of open pull requests by content, should have been part of the process from the start, not added after four fixes had already gone out without it.

A third lesson, smaller but recurring: a security review of one's own new code is not a formality. Across the final batch of patches, a dedicated review caught a real, self-introduced problem in the fix itself in the large majority of cases, not the original bug. That the review process is genuinely useful against one's own code, not just the reported defect, should not be assumed. It has to be exercised deliberately every time.

7 Current Status

Thirteen pull requests from this project are open against the upstream repository as of this report: twelve security and reliability patches and one fully built, live-verified CTF challenge. None have yet been merged. All were opened with a complete process record in their own description: verification performed against the linked issue, tests written, review findings and how they were addressed, and local merge verification against every sibling branch open at the same time. Where a competing contribution from another contributor addressed the same underlying issue, that comparison is documented directly in the relevant pull request.

The four challenges retired in July remain retired. Their design work and the reliability findings behind that decision are preserved in this project's own working notes, in case a future contributor wants to revisit any of them with a different approach.

8 Acknowledgements

Thanks to my mentors, Rishi Mondal and Sanjeev Agarwal, for direct and specific feedback throughout the program, and for taking the reliability concerns raised in July seriously rather than pushing to ship weak content on schedule. Thanks to Venkata Sai Kishore Modalavalasa and Hellen, FinBot's founders, for showing up to every biweekly review in person despite their own busy schedules and for giving clear, final direction at the points in this project that needed it most. Thanks to the OWASP GenAI Security team and the wider FinBot CTF contributor community, whose own quality-assurance work on the codebase's issue backlog made a substantial share of the second half of this project possible.

A Pull Request Index

PR	Title	Underlying issue
558	Cross-Vendor Email, a BOLA/IDOR challenge	New challenge
561	Reject control characters in FinDrive filenames	#360
562	Raise a clear error on missing invoice dates	#290, #291
563	Persist vendor bank account on FinStripe transfers	#328
564	Reject negative simulated balance in FinStripe	#329
565	Reject negative pagination limit in FinStripe	#330
574	Orchestrator no longer confirms a failed payment	#460
575	DNS resolution SSRF fix plus authentication on guardrail webhook endpoints	#535
576	Guardrail payload truncation and missing safety hook	#525
577	Administrative tool override applies both fields	#547
578	Profile share-card cache key corrected	#508
579	Chat assistant guards missing identifier arguments	#418
580	Orchestrator rejects whitespace-only summaries	#459